
Real-Time User Behavior Anomaly Detection Platform

Checkpoint 2 — System Architecture

June 1, 2026

Architecture Overview — Lambda Architecture

The system follows a **Lambda Architecture** with two parallel processing paths:

- **Streaming Layer** — Real-time event processing, feature enrichment, model scoring, and instant decision-making. Maintains “hot” user profiles for sub-millisecond access.
- **Batch Layer** — Heavy computation including full profile rebuilds from historical data, model retraining, model evaluation, and compliance review workflows.

Both layers share common data stores and communicate through Kafka topics, ensuring auditability and fault isolation at every stage.

Streaming Pipeline

Design Principle: Kafka Between Every Stage

Each service boundary is separated by a Kafka topic. This sacrifices a few milliseconds of latency in exchange for:

- **Fault isolation** — a slow or crashed downstream service does not block upstream producers.
- **Auditability** — every transformation stage is logged as a Kafka record; the full event journey is replayable.
- **Buffering** — services consume at their own pace, absorbing traffic spikes.
- **Independent scaling** — each service scales horizontally based on its own throughput needs.

Services

Kafka Topics

Data Stores

#	Service	Responsibility
1	Ingestion Service	Receives raw events, performs minimal validation, publishes to Kafka. Single job: never lose an event, never slow down.
2	Enrichment Service	Consumes raw events, fetches user's hot profile from Redis, computes real-time features (e.g., geo-velocity, amount percentile, tx frequency). Publishes enriched feature vectors.
3	Scoring Service	Consumes enriched events, runs ML model inference, produces anomaly scores. Isolated for independent scaling, deployment, and potential GPU usage.
4	Decision & Alert Service	Consumes scored events, applies business rules to map scores to actions (block / flag / allow / step-up auth), sends notifications to relevant systems and personnel.

Topic	Position	Payload
raw-events	Ingestion → Enrichment	Raw event: account_id, action type, amount, timestamp, location, device, IP, recipient
enriched-events	Enrichment → Scoring	Feature vector: all raw fields + computed features (percentile rank, geo-velocity, session flags, profile deltas)
scored-events	Scoring → Decision	Enriched event + anomaly score + model version + confidence
decisions	Decision → Downstream	Final outcome: action taken (block/flag/allow), reason, score, timestamp. Consumed by batch pipeline, compliance dashboard, and profile update logic.

Redis — Hot Profile Store

- **Purpose:** Sub-millisecond reads during real-time enrichment.
- **Data:** Current user behavioral profile (distributions, known locations/devices/recipients, counters, recent session data).
- **Updated by:** Batch pipeline pushes rebuilt profiles; streaming layer performs lightweight incremental updates (counters, last-seen values).
- **Access pattern:** Key-value lookup by account_id.

PostgreSQL — Cold Profile Store & Data Warehouse

- **Purpose:** Source of truth for all historical data.

- **Data:** Full transaction history, complete user profiles, decision logs, flagged event records, model training datasets.
- **Properties:** ACID transactions, complex analytical queries, joins, reliable backups and replication.
- **Access pattern:** Batch reads/writes for profile rebuilds, retraining data extraction, compliance queries.

Store Interaction Pattern

Batch pipeline reads from PostgreSQL → recomputes full profile distributions → pushes updated profiles to Redis. Streaming pipeline reads from Redis at inference time, writes raw events and decisions back to PostgreSQL for historical record. Redis is effectively a **fast-access cache** of what PostgreSQL computed.

Batch Pipeline

Batch Jobs and Triggers

Job	Trigger	Description
Profile Rebuild	Scheduled (nightly)	Recomputes full user profile distributions from historical transaction data in PostgreSQL. Pushes results to Redis.
Model Retrain	Scheduled (weekly/monthly) + drift trigger	Retrains anomaly detection model on updated labeled data. Triggered early if drift detection flags degradation.
Model Evaluation	Continuous	Monitors model precision, recall, false positive rate. Feeds drift detection; triggers retrain when metrics drop below threshold.
Flagged Event Review	Event-driven (topic decisions)	Populates compliance dashboard for analyst review. Not a scheduled batch job but a near-real-time consumer.

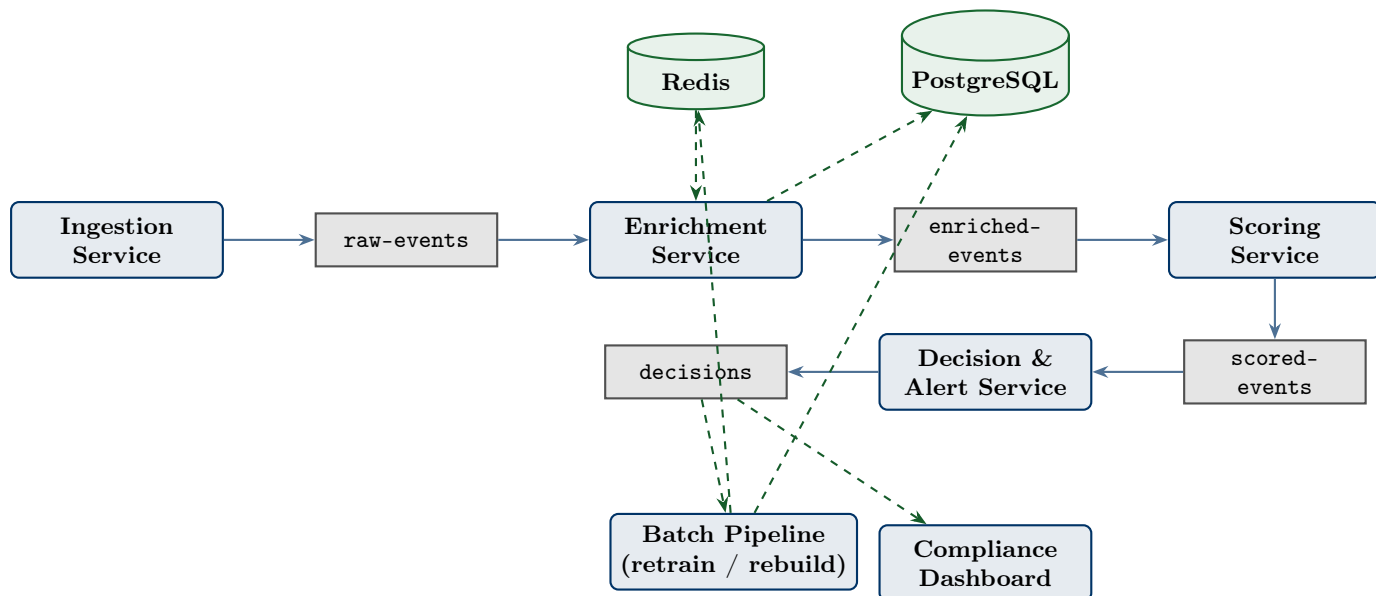
Profile Update Safety

- **Verified-only updates:** Only events that were not flagged as suspicious contribute to profile rebuilds.
- **Decay windows:** Maintain 30-day (short-term) and 6-month (long-term) baselines; compare incoming events against both.
- **Rate-limited drift:** Cap profile shift per time window to prevent gradual profile poisoning attacks.

Service Isolation Principles

- **Fault isolation:** Crash in one service does not propagate; Kafka buffers absorb downstream failures.
- **Independent scaling:** Ingestion scales for throughput; scoring scales for compute; enrichment scales for profile store access.
- **Security boundaries:** Each service has minimal permissions — enrichment reads profiles, scoring runs models, decision service writes alerts.
- **Independent deployment:** New model versions deploy to scoring service without touching ingestion or enrichment.
- **Over-isolation avoided:** Decision and notification are co-located to reduce unnecessary network hops where failure modes and scaling needs align.

Architecture Summary Diagram (Logical)



Next Steps

Design the data schema: raw event structure, enriched feature vector, user profile schema in Redis and PostgreSQL.

Build the data simulator to generate realistic banking events.

Implement services incrementally: ingestion first, then enrichment, scoring, decision.

Define the ML model approach: statistical baselines vs. learned models.

Set up Docker Compose for local development.